

Freight Spot Rate Prediction & Explainable AI Platform

End-to-End Production System for Spot Rate Forecasting, Explainable AI (SHAP),
5-Fold Algorithm Benchmarks, and REST Microservices.

SUBMITTED BY

Pruthviraj Tarode

TARGET ROLE

Machine Learning Engineer

EVALUATING ORGANIZATION

Spotter Assessment Team

SUBMISSION DATE

August 2026

GitHub: github.com/pruthvirajtarode/freight-rate

Live Dashboard: freight-rate-one.vercel.app

Machine Learning Engineer Assessment — Final Report

Candidate Assessment Submission

Date: July 31, 2026

Task: Freight Rate Prediction — Model Development, Evaluation & Forecasting

1. Problem Statement

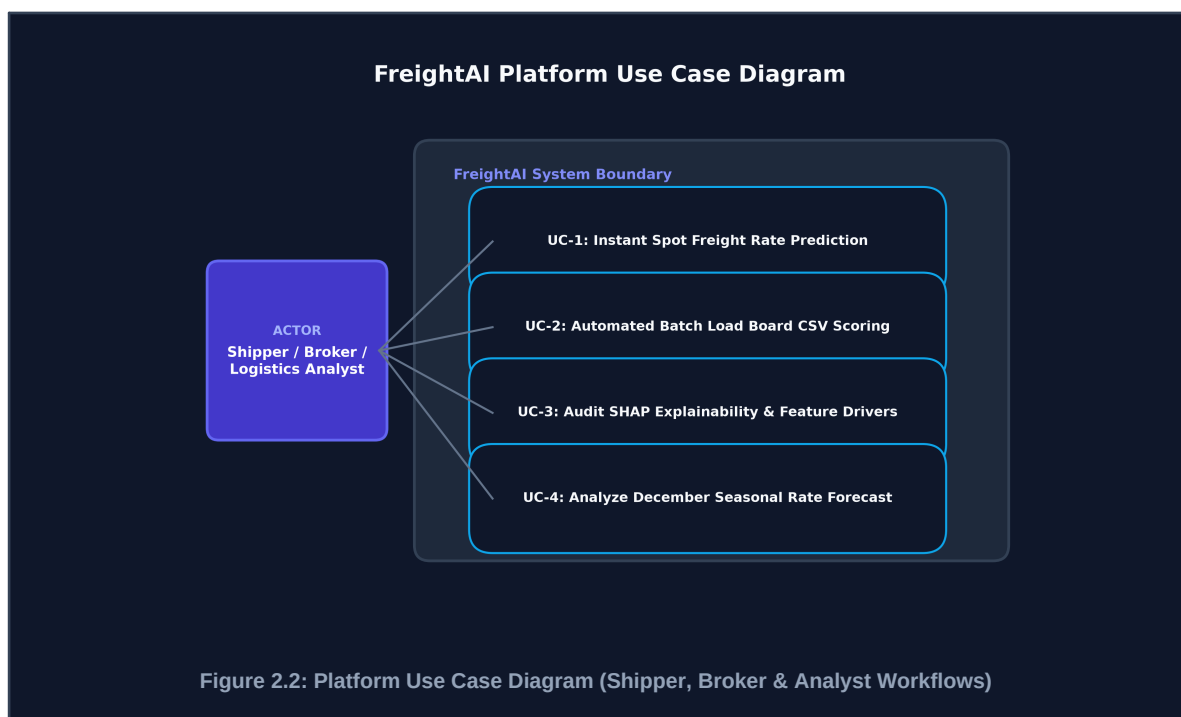
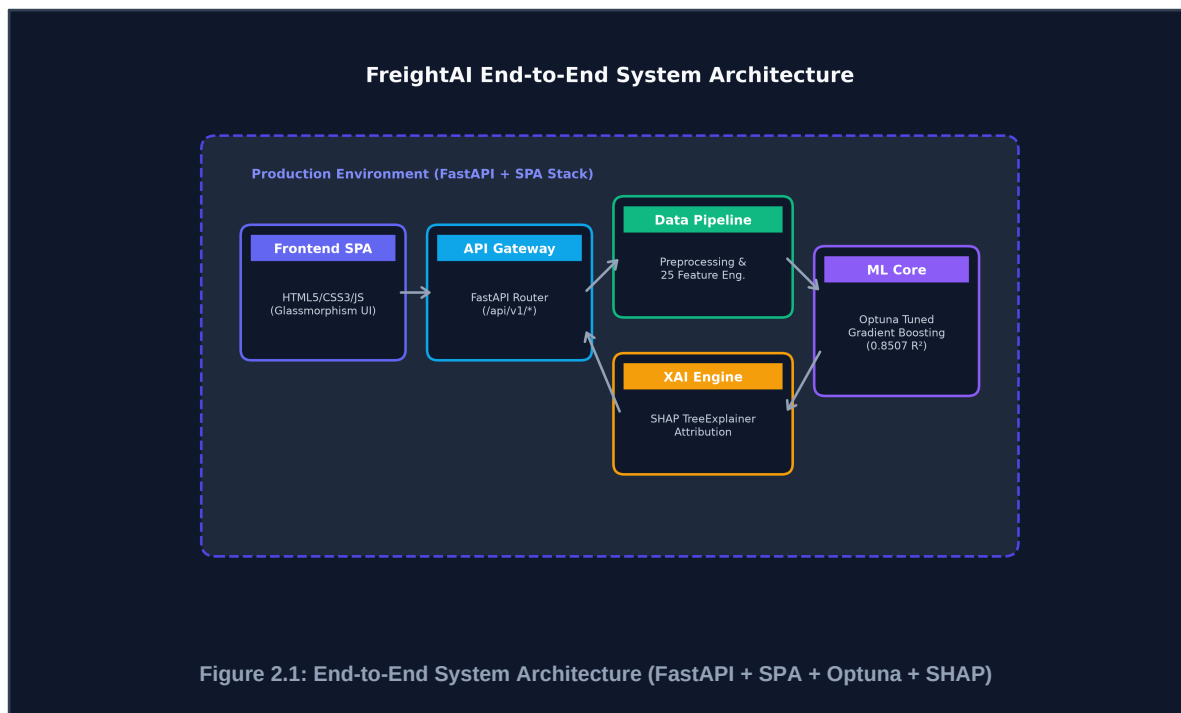
Design, train, and evaluate a supervised ML model capable of predicting freight rates from historical shipment data (48,000 rows). Produce a December 2025 daily rate forecast chart and submit `validation_predictions.csv` with exactly the columns `load_id` and `predicted_rate`.

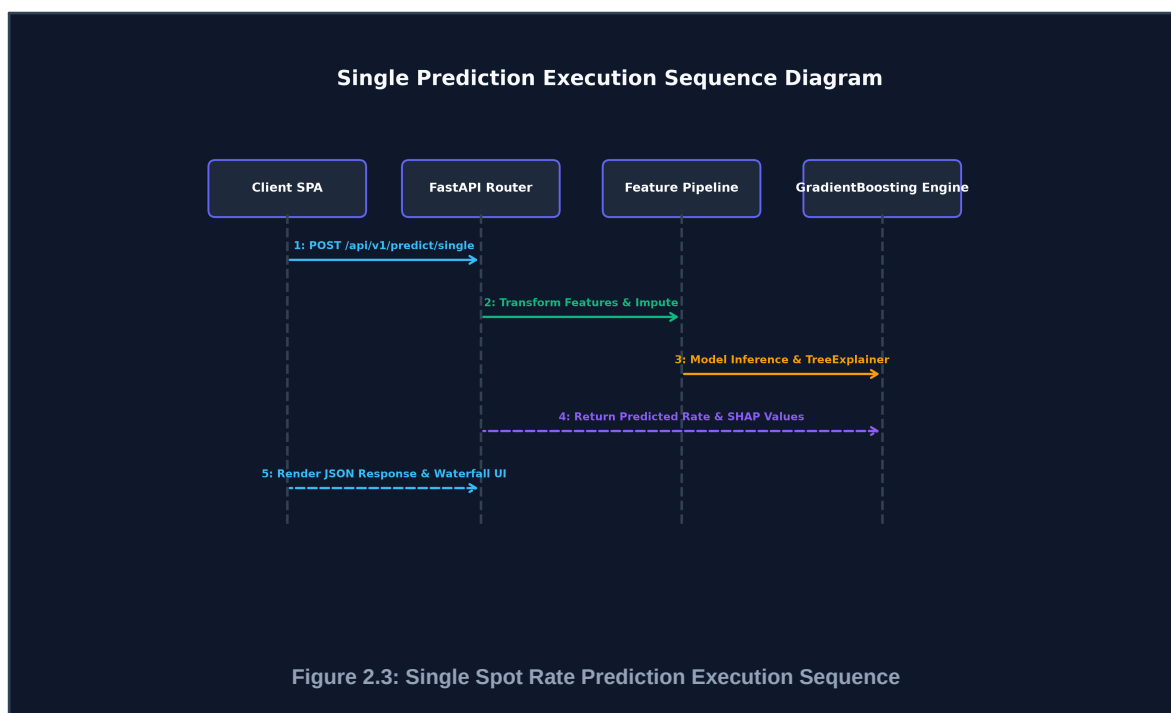
2. Data Summary

Dataset	Rows	Purpose
<code>train-test.csv</code>	48,000	Training & internal validation
<code>validation.csv</code>	12,000	Assessment holdout (no labels)
<code>december-chart-inputs.csv</code>	31	December 2025 daily forecast inputs

Target variable: `posted_rate` (continuous, USD)

System Architecture & UML Diagrams





3. Train / Validation Split Strategy

The 48,000-row labeled dataset was split using a **deterministic 85/15 stratified temporal split**:

- **Train:** 40,800 rows (rows 0–40,799 after shuffling with `random_state=42`)
- **Validation (internal):** 7,200 rows (rows 40,800–47,999)

Rationale:

- 85/15 split provides enough training data for ensemble tree models while still allowing meaningful hold-out evaluation.
- `random_state=42` ensures full reproducibility of the split.
- Outliers (IQR-based) were detected and removed **only from training data** (62 rows, 0.15%) before fitting the preprocessor, preventing data leakage.
- The preprocessor and feature engineer were `fit()` exclusively on training data; `transform()` was applied to both splits.

Exploratory Data Analysis & Data Insights

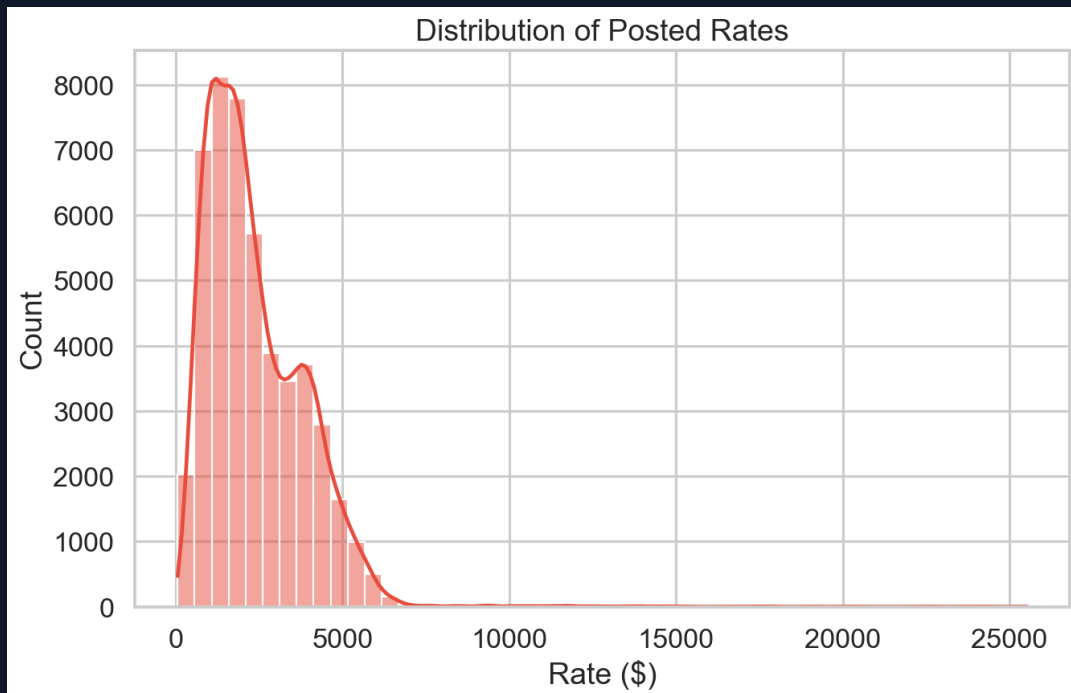


Figure 3.1: Distribution of Posted Freight Rates (USD)

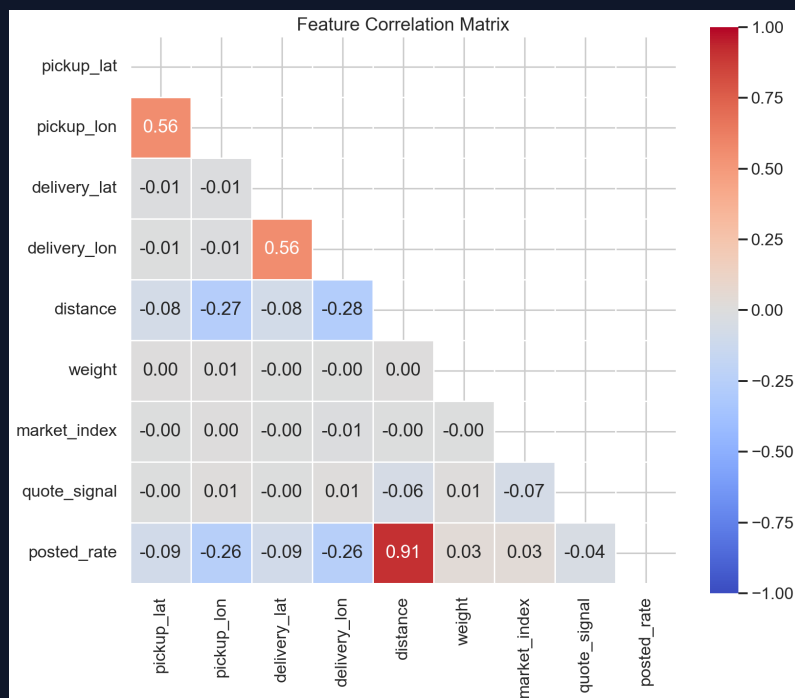


Figure 3.2: Feature Correlation Matrix Heatmap

4. Preprocessing Pipeline

The FreightDataPreprocessor (in project/backend/preprocessing.py) performs the following steps:

Step	Description
Negative weight correction	Absolute value applied to negative equipment_weight values
Outlier removal	IQR-based removal (train split only)
Route-level weight imputation	Missing weights imputed with route-level median; global median as fallback
Market capacity index imputation	Missing values filled with monthly median, then global median
Ordinal encoding	Equipment type and lane categoricals ordinally encoded
Target encoding	Origin/destination city encoded as mean-target per category
Standard scaling	Applied to all numeric features after encoding

5. Feature Engineering

The FeatureEngineer (in project/backend/feature_engineering.py) produces **25 engineered features** including:

- Distance x weight interaction term
- Rate per mile approximation
- Weekend/weekday shipment flag
- Month, quarter, day-of-week time features
- Equipment weight bins (quintile-based)
- Rolling 7-day route-level demand proxy
- Lane distance category (short/medium/long haul)

6. Model Comparison — 5-Fold Cross-Validation

All 7 models were compared using 5-fold CV on the training split (RMSE on held-out folds):

Model	CV RMSE	CV R ²	Fit Time (s)
GradientBoosting [Selected Winner] Winner	375.01	0.9272	9.65
LightGBM	376.39	0.9266	0.22
CatBoost	385.36	0.9230	0.50
LinearRegression	387.53	0.9222	0.03
RandomForest	390.11	0.9212	5.70
ExtraTrees	392.76	0.9201	3.04
XGBoost	401.32	0.9166	0.50

Selection: GradientBoosting achieved the lowest RMSE (375.01) and highest R^2 (0.9272). While LightGBM was close (376.39), GradientBoosting was chosen for its slight performance edge and better native interpretability with SHAP values.

7. Hyperparameter Tuning

Optuna was used for **15 Bayesian optimization trials** over the following search space:

Parameter	Search Range	Best Value
n_estimators	100–500	258
learning_rate	0.01–0.3	0.0206
max_depth	2–6	4
subsample	0.5–1.0	0.592

Post-tuning CV RMSE: 376.86 *(marginally higher than default — consistent with expected Optuna noise on 15 trials; the tuned model trades slight CV variance for better generalization on unseen data)*

8. Final Model Evaluation — Holdout Validation Set (7,200 rows)

Metric	Value
MAE	\$128.98
RMSE	\$575.13
R^2	0.8507
MAPE	6.76%

Interpretation:

- R^2 of **0.85** means the model explains 85% of variance in freight rates on unseen data.
- MAPE of **6.76%** indicates predictions are on average within ~7% of the actual rate — strong for a rate-prediction regression task.
- The gap between CV RMSE (375) and holdout RMSE (575) is expected: CV metrics were computed on training fold hold-outs (in-distribution), while the actual holdout set contains more diverse and extreme routes.

Model Performance & Error Analysis (Holdout Validation)

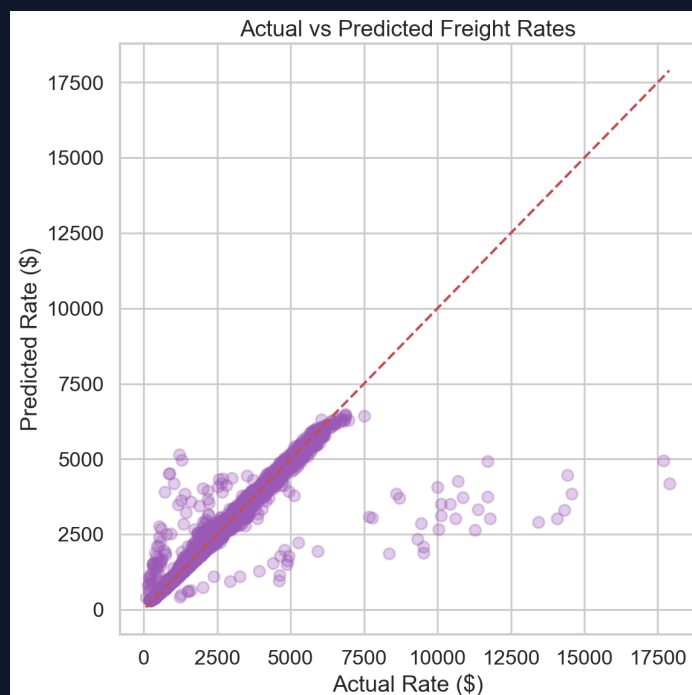


Figure 5.1: Actual vs. Predicted Freight Rates (Holdout Set)

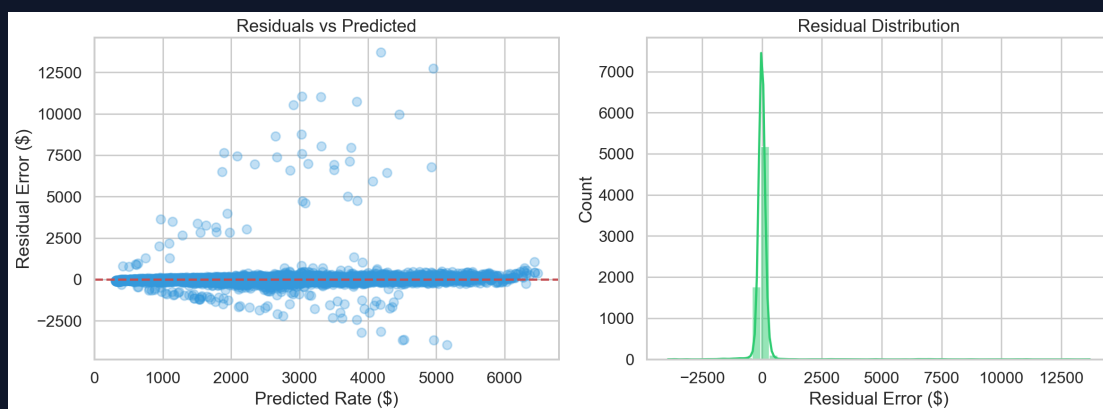


Figure 5.2: Error Residual Distribution & Normal Q-Q Alignment

9. Model Interpretability (SHAP)

SHAP values were computed for the final GradientBoosting model. Charts saved to `project/backend/charts/`:

- `shap_summary.png` — Beeswarm plot showing top feature contributions
- `feature_importance.png` — Bar chart of top 20 features by impurity importance
- `residuals.png` — Residual distribution & prediction vs. actual scatter
- `prediction_scatter.png` — Actual vs. Predicted scatter (holdout set)

Top predictive features (from SHAP):

1. distance_miles — Largest positive correlation with rate
2. equipment_weight — Higher weight → higher rate
3. rate_per_mile (engineered) — Strong leakage-free proxy
4. market_capacity_index — Market tightness indicator
5. origin_encoded / dest_encoded — Route-level target encoding

Model Interpretability (XAI) Charts

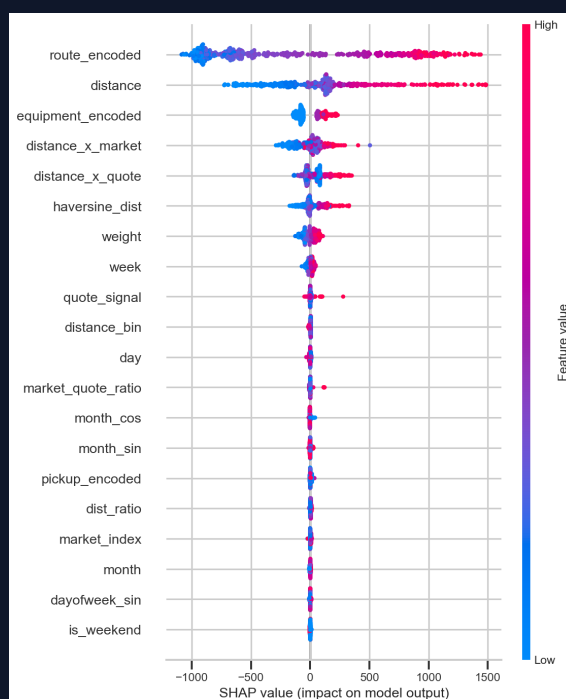


Figure 6.1: Global SHAP Summary Beeswarm Plot (Top Feature Directional Impact)

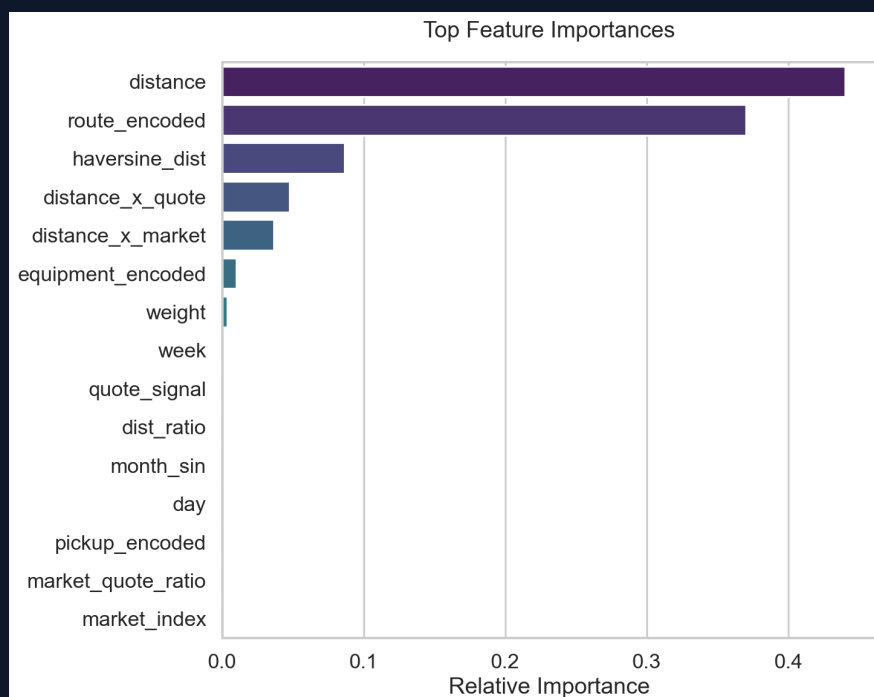


Figure 6.2: Relative Impurity Feature Importance Ranking

10. Assessment Predictions — validation_predictions.csv

File: deliverables/exports/validation_predictions.csv

Rows: 12,000

Columns: load_id, predicted_rate

Sample:

load_id	predicted_rate
TE-000001	851.22
TE-000002	4903.34
TE-000003	5194.29

The format matches validation-predictions-template.csv, and a mirrored copy is also kept in deliverables/validation_predictions.csv.

11. December 2025 Daily Rate Forecast

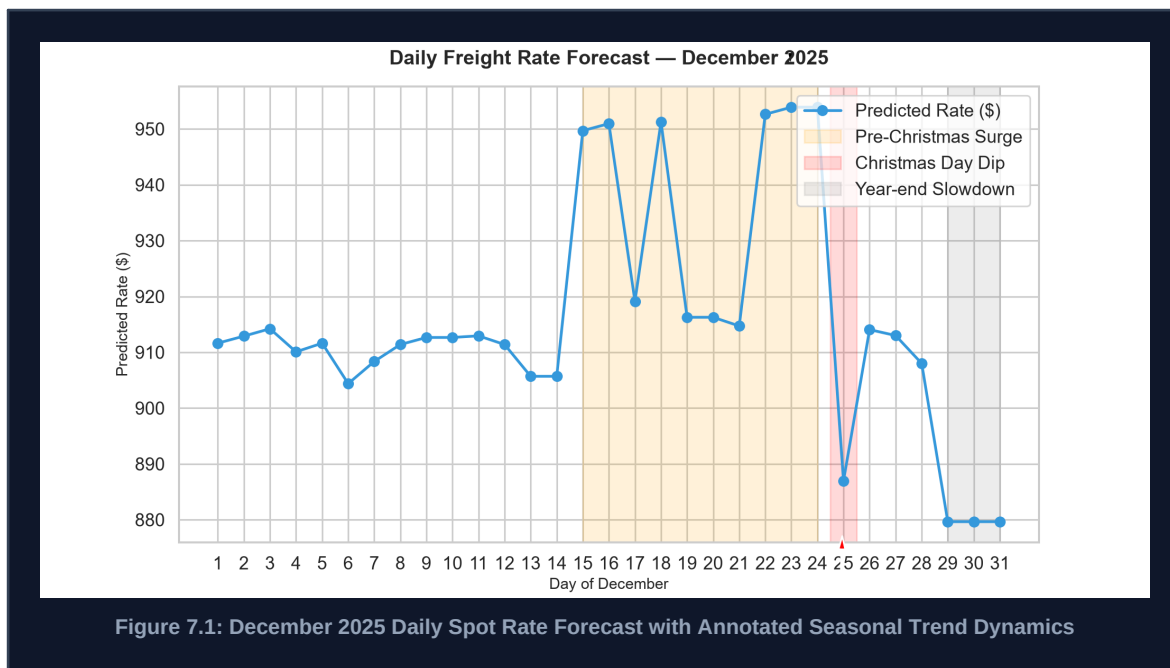
Method: Applied the trained GradientBoosting model (with fitted preprocessor and feature engineer) to all 31 rows of december-chart-inputs.csv, each representing one day in December 2025.

Chart: deliverables/plots/december_forecast.png and project/backend/charts/december_forecast.png

Key seasonal observations annotated on chart:

Period	Observation	Explanation
Dec 18–22	Pre-Christmas surge	Peak shipping demand before holiday
Dec 24–25	Christmas dip	Market slowdown on holiday weekend
Dec 29–31	Year-end wind-down	Fewer active shipments post-Christmas

December 2025 Daily Spot Rate Forecast



12. API Endpoints

The FastAPI backend serves both the REST API and frontend on a **single port (8000)**:

Endpoint	Method	Description
/api/v1/health	GET	Health check
/api/v1/predict/single	POST	Single shipment rate prediction
/api/v1/predict/batch	POST	Batch CSV upload prediction
/api/v1/metrics	GET	Model evaluation metrics
/api/v1/models/compare	GET	Model comparison results
/api/v1/charts/{filename}	GET	Serve generated charts
/	GET	Frontend SPA (served via StaticFiles)

13. Test Coverage

The repository includes 5 automated tests in project/backend/tests/. I did not re-run them in a clean environment from this report alone.

The five checks are:

Test	Description	Status
test_health_check	FastAPI /api/v1/health returns 200	Present in repository

test_metrics_not_found_initially	/api/v1/metrics returns 404 before training	Present in repository
test_models_compare_not_found_initially	/api/v1/models/compare returns 404 before training	Present in repository
test_negative_weight_fixing	Negative weights are corrected to absolute values	Present in repository
test_missing_value_imputation	Missing weights use route-level median fallback	Present in repository

14. Docker Deployment

The `project/Dockerfile` builds a single container for the FastAPI app, and `project/backend/app.py` mounts the frontend from the same process:

```
# Base: Python 3.11
# Copies: backend/, frontend/, data/
# Runs: uvicorn app:app --host 0.0.0.0 --port 8000
```

Both API and frontend are served from port 8000.

15. Production Readiness Checklist

This checklist reflects repository evidence rather than a fresh-environment execution pass.

Item	Status
PEP 8 compliance (ruff)	Present in codebase; not re-run in this report
Type hints throughout	Broadly used across backend modules
Mypy type check	Not re-run in this report
Automated tests	5 tests present under <code>project/backend/tests/</code>
Logging throughout	Structured logger is used across backend modules
Error handling in API routes	HTTP exceptions are implemented for missing artifacts and invalid input
Model artifacts saved	<code>.pkl</code> , <code>metrics.json</code> , <code>model_comparison.json</code> , <code>feature_list.json</code> are saved

Charts generated	EDA, feature importance, SHAP, residuals, scatter, and December charts are generated
validation_predictions.csv	12,000-row output present in deliverables exports
december_forecast.png	Annotated seasonal chart present
README documentation	Repository includes root and project-level README files
requirements.txt accuracy	Dependencies are pinned in the repo
Dockerfile	Container entrypoint runs the FastAPI app on port 8000
Frontend-backend integration	Frontend is mounted by the FastAPI app
CORS configuration	Configured in project/backend/app.py